DEVALIVE

Database Architecture Document (DAD)

Version: 1.0

Prepared By: Tanmay Bonde

Database Type: MongoDB Atlas

Document Status: Approved

---

1. Introduction

Purpose

This document defines the complete database architecture for DevAlive.

It describes:

- Collections
- Document Structures
- Relationships
- Indexing Strategy
- Data Lifecycle
- Monitoring Data Flow
- Notification Data Storage
- Analytics Data Storage
- Future Scalability Considerations

The objective is to provide a production-ready database blueprint that supports the DevAlive SaaS platform.

---

2. Database Overview

Database Name

```
devalive
```

Database Type

```
MongoDB Atlas
```

Architecture Style

```
Document Oriented Database
```

ODM

```
Mongoose
```

---

3. Collection Overview

DevAlive MVP consists of 5 primary collections.

```
Users
Projects
MonitoringLogs
Notifications
UserSettings
```

Future Collections

```
Teams
Subscriptions
Billing
APIKeys
AuditLogs
MonitoringRegions
```

---

4. Entity Relationship Overview

```
User
 │
 ├── Projects
 │      │
 │      ├── Monitoring Logs
```

```
  │      │
  │      └── Notifications
  │
  └── Settings
```

Relationship Types

```
User → Projects
1 : Many

Project → Monitoring Logs
1 : Many

Project → Notifications
1 : Many

User → Settings
1 : 1
```

---

5. Users Collection

Collection Name

```
users
```

Purpose

Stores authentication and account information.

Document Structure

```
{
  _id: ObjectId,

  fullName: String,

  email: String,

  password: String,
```

```
  emailVerified: Boolean,

  profileImage: String,

  accountStatus: String,

  role: String,

  lastLoginAt: Date,

  createdAt: Date,

  updatedAt: Date
}
```

Field Definitions

fullName

```
Required
Maximum 100 Characters
```

email

```
Required
Unique
Indexed
```

password

```
Bcrypt Hashed Password
```

accountStatus

```
active
suspended
deleted
```

role

```
user
admin
```

Indexes

```
email: 1
```

---

6. Projects Collection

Collection Name

```
projects
```

Purpose

Stores monitored projects.

Document Structure

```
{
  _id: ObjectId,

  userId: ObjectId,

  projectName: String,

  description: String,

  endpointUrl: String,

  projectType: String,

  monitoringInterval: Number,

  monitoringEnabled: Boolean,

  currentStatus: String,
```

```
  lastCheckedAt: Date,

  lastSuccessAt: Date,

  lastFailureAt: Date,

  totalChecks: Number,

  successfulChecks: Number,

  failedChecks: Number,

  uptimePercentage: Number,

  averageResponseTime: Number,

  createdAt: Date,

  updatedAt: Date
}
```

Field Definitions

projectType

```
website
api
backend
health-endpoint
```

monitoringInterval

```
Minutes

5
10
15
30
60
```

currentStatus

```
online
offline
degraded
unknown
```

Indexes

```
userId: 1

currentStatus: 1

monitoringEnabled: 1
```

---

7. Monitoring Logs Collection

Collection Name

```
monitoringlogs
```

Purpose

Stores every monitoring execution.

This collection becomes the largest collection in the system.

Document Structure

```
{
  _id: ObjectId,

  userId: ObjectId,

  projectId: ObjectId,

  status: String,

  httpStatusCode: Number,
```

```
  responseTime: Number,

  errorMessage: String,

  checkedAt: Date,

  createdAt: Date
}
```

Field Definitions

status

```
success
failure
timeout
error
```

httpStatusCode

Examples

```
200
201
404
500
503
```

responseTime

```
Milliseconds
```

Indexes

```
projectId: 1

userId: 1
```

```
checkedAt: -1

status: 1
```

Retention Strategy

Monitoring logs can grow rapidly.

MVP:

```
Store Forever
```

Future:

```
Archive Logs Older Than 6 Months
```

---

8. Notifications Collection

Collection Name

```
notifications
```

Purpose

Stores alert history.

Document Structure

```
{
  _id: ObjectId,

  userId: ObjectId,

  projectId: ObjectId,

  notificationType: String,
```

```
  title: String,

  message: String,

  deliveryStatus: String,

  sentAt: Date,

  createdAt: Date
}
```

Notification Types

```
downtime_alert

recovery_alert

monitoring_failure

account_notification
```

Delivery Status

```
pending

sent

failed
```

Indexes

```
userId: 1

projectId: 1

sentAt: -1
```

---

9. User Settings Collection

Collection Name

```
usersettings
```

Purpose

Stores user preferences.

Document Structure

```
{
  _id: ObjectId,

  userId: ObjectId,

  emailNotifications: Boolean,

  downtimeAlerts: Boolean,

  recoveryAlerts: Boolean,

  defaultMonitoringInterval: Number,

  timezone: String,

  createdAt: Date,

  updatedAt: Date
}
```

Relationship

```
One User
One Settings Document
```

Indexes

```
userId: 1
```

---

10. Data Flow Architecture

User Registration

```
User
 ↓
Users Collection
 ↓
UserSettings Collection
```

Project Creation

```
User
 ↓
Projects Collection
 ↓
Monitoring Activated
```

Monitoring Execution

```
Scheduler
 ↓
Project
 ↓
Endpoint Request
 ↓
Monitoring Log
 ↓
Project Statistics Updated
```

Failure Detection

```
Monitoring Log
 ↓
Failure Identified
 ↓
Notification Created
 ↓
Email Sent
```

---

11. Analytics Data Strategy

Instead of calculating analytics from scratch every request:

Store aggregated values inside Projects.

Examples:

```
uptimePercentage

totalChecks

successfulChecks

failedChecks

averageResponseTime
```

Benefits:

- Faster dashboard loading
- Lower database load
- Better scalability

---

12. Monitoring Data Lifecycle

Monitoring Request

↓

Monitoring Log Created

↓

Project Statistics Updated

↓

Failure Detection

↓

Notification Triggered

↓

Dashboard Updated

---

13. Scalability Strategy

Current MVP

```
Single MongoDB Cluster
```

Expected Capacity

```
Thousands Of Users

Millions Of Monitoring Logs
```

Future Scaling

```
MongoDB Sharding

Archive Collections

Analytics Collections

Queue Based Processing
```

---

14. Security Strategy

Sensitive Data

```
Passwords
```

Stored As

```
Bcrypt Hash
```

Never Store

```
Plain Passwords
```

Access Control

```
User Can Only Access Own Projects

User Can Only Access Own Logs

User Can Only Access Own Notifications
```

---

15. Database Naming Standards

Collections

```
lowercase
plural
```

Examples

```
users

projects
```

```
monitoringlogs

notifications

usersettings
```

Fields

```
camelCase
```

Examples

```
projectName

endpointUrl

monitoringInterval

responseTime
```

---

16. Future Database Extensions

Phase 2

```
Teams Collection

Organization Collection
```

Phase 3

```
Subscriptions Collection

Billing Collection
```

Phase 4

```
API Keys Collection

Webhooks Collection
```

Phase 5

```
Monitoring Regions Collection

Global Monitoring Nodes Collection
```

---

17. Database Architecture Summary

The DevAlive database architecture is designed to support:

- User Management
- Project Monitoring
- Monitoring History
- Analytics
- Notifications
- Future SaaS Scaling

Core Collections:

1. Users
2. Projects
3. MonitoringLogs
4. Notifications
5. UserSettings

The architecture prioritizes:

- Scalability
- Query Performance
- Analytics Efficiency
- Maintainability
- Future Enterprise Expansion